

iFody COSC 310 - Deployment and Handover Guide

GitHub Username - Student Number

- udeluca - 75350892
- jhlbolsonaro - 82056300
- JoshL940508 - 55346662
- treyhuculak - 44925055
- LostInHustle - 14288765

Youtube link for our presentation

- <https://youtu.be/qb5XH1fy3Ug>

Contribution Statement (M4):

- Trey: 20%
- Aaron: 20%
- Joao: 20%
- Umberto: 20%
- Josh: 20%

Contribution Statement (Total Project):

- Trey: 20%
- Aaron: 20%
- Joao: 20%
- Umberto: 20%
- Josh: 20%

Purpose and Scope

This guide is designed for a TA or new maintainer to deploy, run, and maintain the iFody project from scratch using Docker Compose.

It covers:

- End-to-end installation and startup steps.
- Required dependencies with version details.
- Environment variable setup (including a .env file).
- Ongoing maintenance requirements (accounts, data, and external services).

1. System Overview

The project runs as 3 Docker services:

- backend: FastAPI application (Python).
- frontend: React + Vite development server.
- ollama: local LLM runtime (llama3.2:3b) used by chatbot features.

Default URLs after startup:

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:8000>

- Ollama API: <http://localhost:11434>

2. Dependencies and Version Requirements

2.1 Host Tools

Install the following on the host machine:

- Docker Desktop (recommended latest stable).
- Docker Compose v2 (included with modern Docker Desktop).
- Git (for cloning and updates).
- Optional for non-Docker testing: Python 3.12 and Node 20.

2.2 Runtime Images Used by This Repository

From project files:

- Backend runtime image: python:3.12-slim
- Frontend runtime image: node:20-alpine
- Ollama runtime image: ollama/ollama

2.3 Application Libraries

- Backend libraries are pinned in requirements.txt (FastAPI 0.128.0, uvicorn 0.40.0, requests 2.33.1, pytest 9.0.2, etc.).
- Frontend libraries are defined in src/frontend/package.json (React 19.2.4, React Router 7.13.2, Vite 8.0.1, ESLint 9.39.4).

3. Step-by-Step Deployment (Docker, Start to Finish)

Step 1 - Get the Source Code

If you do not already have the project locally:

```
git clone https://github.com/treyhuculak/iFody-COSC-310-Project.git
cd iFody-COSC-310-Project
```

If you already have it:

```
git pull
```

Step 2 - Create the .env File

A template is provided at .env.example.

Create .env in the project root:

PowerShell:

```
Copy-Item .env.example .env
```

macOS/Linux:

```
cp .env.example .env
```

Edit .env and set values. Minimum recommended content:

```
PAYPAL_CLIENT_ID=ARaUVIeGUY05Lnc7r_LeIxsfwg0Hy2xjk8cYaJO_mNrqyNvDr8kdUICA8sgSRTHoXGTMiPjFbEJu9c
PAYPAL_CLIENT_SECRET=ELQSB_Xwb6Xx3jqn1aCTAj1Zqd-
zain4kwE9aAYS4y9zLkNhnMSVYu6tAVUU_XDJ0ACxqpF5J8g4Zir0
PAYPAL_BASE_URL=https://api-m.sandbox.paypal.com/
FRONTEND_URL=http://localhost:3000/

EMAIL_ADDRESS=ifodyproject@gmail.com
EMAIL_PASSWORD=iFody2026@123
APP_PASSWORD=ozdu gobi qaxf bleb

OLLAMA_URL=http://ollama:11434

VITE_API_URL=/api
VITE_POPULAR_RESTAURANTS_ENDPOINT=/restaurants/popular
VITE_RECENT_RESTAURANTS_ENDPOINT=/restaurants/recent
```

Notes:

- PAYPAL_CLIENT_ID and PAYPAL_CLIENT_SECRET are required for PayPal transaction flows.
- APP_PASSWORD is the Gmail app password used by email notifications.
- FRONTEND_URL is used in PayPal return/cancel links.
- OLLAMA_URL should remain <http://ollama:11434> when running in Docker Compose.

Step 3 - Build and Start Containers

From project root:

```
docker compose up --build
```

What this does:

- Builds backend and frontend images.
- Starts backend, frontend, and ollama.
- Mounts ./data into containers for JSON-backed persistence.
- Pulls/warms the Ollama model (llama3.2:3b) on startup scripts.

First run can take longer because model/image downloads are large.

Step 4 - Verify Services

After startup, verify:

- Frontend loads: <http://localhost:3000>
- Backend responds: <http://localhost:8000>
- Backend root should return JSON with "Hello World".

Optional terminal checks:

```
docker compose ps
docker compose logs backend --tail=100
docker compose logs frontend --tail=100
docker compose logs ollama --tail=100
```

Step 5 - Basic Functional Smoke Test

1. Open frontend and log in using a seeded test account (see Section 4.2).
2. Browse restaurants and menu items.
3. For owner role account, verify owner portal access.
4. For customer account, verify cart/order flow.
5. If PayPal is configured, test payment initiation/capture.

Step 6 - Stop the System

Stop and remove running containers:

```
docker compose down
```

To also remove local images built for this project:

```
docker compose down --rmi local
```

4. Maintenance Requirements

4.1 Day-to-Day Operations

Common commands:

```
docker compose up -d
docker compose logs -f backend
docker compose restart backend
docker compose down
```

4.2 Account Credentials (Seed Data)

User data is stored in data/user_dbjson.

Common seeded accounts:

- Customer:
 - Email: testcustomer@123.com
 - Password: Test@123
- Restaurant Owner:
 - Email: testowner@123.com
 - Password: Test@123
- Administrator:
 - Email: testadmin@123.com
 - Password: Test@123

Maintenance recommendation:

- Rotate/remove default credentials before production/demo handoff.
- Avoid committing real personal credentials in data files.

4.3 Data Management Procedures

This project currently uses JSON files under the data directory as the datastore.

Primary data files include:

- data/restaurants.json
- data/orders.json
- data/payment.json
- data/transaction.json
- data/notification.json
- data/delivery.json
- data/offers.json
- data/weekly_offers.json
- data/user_db.json
- data/tax_rates.json

Important behavior:

- Docker Compose mounts ./data into backend and ollama containers.
- Data persists across container restarts.
- This storage model is suitable for local/single-instance use, not horizontal scaling.

Backup procedure (PowerShell example):

```
New-Item -ItemType Directory -Force -Path .\backups | Out-Null
Compress-Archive -Path .\data\* -DestinationPath .\backups\data-backup.zip -Force
```

Restore procedure:

1. Stop containers: docker compose down
2. Restore files into ./data
3. Start again: docker compose up -d

Reset procedure for a clean test run:

1. Stop containers.
2. Replace data files with known baseline copies.
3. Start containers.

4.4 External API and Service Configuration

PayPal

Used by backend transaction endpoints. Required .env keys:

- PAYPAL_CLIENT_ID
- PAYPAL_CLIENT_SECRET
- PAYPAL_BASE_URL
- FRONTEND_URL

Recommended setup:

- Use PayPal Sandbox credentials for TA evaluation.
- Keep PAYPAL_BASE_URL as sandbox unless intentionally moving to production.
- Use the following personal PayPal Developer Sandbox account when paying with the PayPal option(login after clicking approve link within the app):
 - sb-r5mas50346263@personal.example.com
 - l&+tax1z

Email (Gmail SMTP)

Used by EmailService for notifications. Required .env keys:

- EMAIL_ADDRESS
- APP_PASSWORD

Notes:

- SMTP server is hardcoded to smtp.gmail.com:587.
- APP_PASSWORD should be a Google app password, not your normal login password.

Ollama / Chatbot

Chatbot service posts requests to OLLAMA_URL.

- In Docker Compose, service name routing is used (<http://ollama:11434>).
- Startup script pulls model llama3.2:3b automatically.

If chatbot responses fail:

- Check ollama container logs.
- Confirm model pull completed.
- Confirm OLLAMA_URL is correct.

5. Troubleshooting

Problem: ModuleNotFoundError when running pytest locally

Example: ModuleNotFoundError: No module named 'requests'

Cause:

- Local virtual environment is missing packages.

Fix:

```
python -m pip install -r requirements.txt
pytest
```

Problem: Frontend cannot reach backend

Checks:

- Ensure backend is running on 8000.
- Ensure frontend proxy points to backend.
- Confirm VITE_API_URL is /api unless custom routing is intentional.

Problem: PayPal start/capture errors

Checks:

- Verify PAYPAL_CLIENT_ID and PAYPAL_CLIENT_SECRET.
- Verify PAYPAL_BASE_URL is sandbox URL for sandbox credentials.
- Inspect backend logs for API error payloads.

Problem: Email not sending

Checks:

- Verify EMAIL_ADDRESS and APP_PASSWORD.
- Confirm Gmail app password is valid.
- Check backend logs for SMTP errors.

6. Handover Checklist (TA Ready)

Before submitting or handing off:

- .env.example is present and complete.
- .env is configured locally (not committed with secrets).
- docker compose up --build succeeds from project root.
- Frontend and backend URLs are reachable.
- Seed login accounts are documented.
- PayPal and email setup status is documented (configured or intentionally omitted).
- Data backup/restore procedure is documented and tested.
- Ollama model pulls successfully, and the chatbot responds to test messages.

7. Optional Non-Docker Run (Reference)

If needed for debugging only:

- Backend: fastapi dev src/backend/main.py
- Frontend:
 - cd src/frontend
 - npm install
 - npm run dev

Docker Compose remains the primary deployment method for project handover.